

Medición de Rendimiento en el Conteo de Frecuencia de Palabras

Comparación entre una implementación secuencial
y una implementación paralela con POSIX threads

Análisis de tiempo de ejecución, speedup y efficiency

Integrantes del grupo

Pablo Cabrera

Eliazar Canastuju

Diego Ramírez

André Pivaral

Hoja de Trabajo No. 1

11 de agosto de 2026

Índice

1. Introducción y objetivos	2
2. Entorno experimental	2
2.1. Justificación del archivo complementario	3
3. Diseño de las implementaciones	3
3.1. Estructura de datos común	3
3.2. Versión secuencial	3
3.2.1. Modificaciones respecto al Corto 02	4
3.3. Versión paralela	4
3.3.1. Estrategia de descomposición	4
3.3.2. División del texto sin fragmentar palabras	5
3.3.3. Tablas privadas por hilo	5
3.3.4. Sincronización y reducción	6
4. Metodología de medición	6
4.1. Delimitación de la región medida	6
4.2. Verificación de correctitud	7
4.3. Justificación de las cinco ejecuciones	7
5. Resultados experimentales	8
5.1. Tiempos de ejecución	8
5.2. Cálculo del <i>speedup</i>	8
5.3. Cálculo de la <i>efficiency</i>	9
5.4. Tabla final de resultados	10
6. Análisis de resultados	10
6.1. ¿Con qué cantidad de hilos se obtuvo el mayor <i>speedup</i> ?	10
6.2. ¿Con qué configuración se obtuvo la mayor <i>efficiency</i> ?	10
6.3. ¿El <i>speedup</i> aumentó proporcionalmente?	11
6.4. ¿La <i>efficiency</i> aumentó o disminuyó?	11
6.5. Factores que pudieron afectar el tiempo de ejecución	11
6.6. Fuentes de <i>overhead</i> identificadas	12
6.7. ¿Agregar más hilos garantiza siempre un mejor rendimiento?	12
7. Conclusiones	13
8. Reproducción de los resultados	13

1. Introducción y objetivos

El presente informe documenta la medición y comparación del rendimiento de dos implementaciones del algoritmo de conteo de frecuencia de palabras: una **secuencial** y una **paralela**. Ambas parten del programa desarrollado en la actividad Corto 02 y fueron instrumentadas para medir el tiempo empleado específicamente durante la fase de conteo.

Los objetivos concretos del trabajo son:

1. Instrumentar ambas versiones para medir el tiempo de la fase de conteo, asegurando que la medición cubra exactamente la misma porción del algoritmo en las dos.
2. Ejecutar cinco corridas por configuración (secuencial, 2 hilos y 4 hilos) sobre el mismo archivo de texto y obtener el tiempo promedio.
3. Calcular el *speedup* y la *efficiency* de cada configuración paralela.
4. Analizar el comportamiento del rendimiento al variar la cantidad de hilos e identificar las fuentes de *overhead*.

Criterio metodológico fundamental. Un valor de *speedup* solo tiene significado si ambas versiones calculan exactamente lo mismo y si el cronómetro cubre el mismo trabajo en las dos. Por esta razón, antes de reportar cualquier medición se verificó la equivalencia de las salidas (sección 4.2) y se delimitó con precisión la región medida (sección 4.1).

2. Entorno experimental

Todas las mediciones se realizaron en el mismo equipo y bajo las mismas condiciones, sin otras aplicaciones de carga activa.

Cuadro 1: Características del entorno de pruebas

Elemento	Valor
Procesador	Apple M5 (arquitectura ARM64)
Núcleos	10 lógicos: 4 de rendimiento (P) + 6 de eficiencia (E)
Sistema operativo	macOS 26.5.2
Compilador	Apple clang 21.0.0
Banderas de compilación	-O2 -Wall (secuencial) -O2 -Wall -pthread (paralela)
Modelo de paralelismo	POSIX threads (pthreads), memoria compartida
Reloj utilizado	clock_gettime(CLOCK_MONOTONIC), resolución de nanosegundos
Archivo principal	archivo.txt: 24530 bytes, 4589 palabras, 904 palabras distintas
Archivo complementario	archivo_grande.txt: 981240 bytes, 183560 palabras, 904 distintas

La arquitectura heterogénea del procesador es relevante para la interpretación de los resultados: los núcleos P y E **no tienen el mismo desempeño**, de modo que el planificador del sistema operativo puede asignar hilos a núcleos de distinta velocidad.

2.1. Justificación del archivo complementario

El enunciado exige emplear el mismo archivo de texto en todas las pruebas, condición que se cumple dentro de cada experimento. Se incorporó adicionalmente un segundo experimento sobre un archivo mayor —construido repitiendo cuarenta veces el mismo texto— por dos motivos:

- Con 24530 bytes los tiempos son del orden de milisegundos y la variabilidad del sistema operativo llega a duplicar la medición entre corridas.
- Comparar ambos tamaños permite **aislar el efecto del tamaño del problema** sobre el *overhead*, sin modificar una sola línea de código. Como se verá en la sección 6, este contraste resultó ser el hallazgo más informativo del trabajo.

3. Diseño de las implementaciones

3.1. Estructura de datos común

Las frecuencias se almacenan en un arreglo dinámico de pares (*palabra*, *frecuencia*) que duplica su capacidad al llenarse:

```
1 typedef struct {
2     char *palabra;      /* copia propia de la palabra */
3     size_t frecuencia;  /* numero de apariciones */
4 } EntradaFrecuencia;
5
6 typedef struct {
7     EntradaFrecuencia *entradas;
8     size_t cantidad;    /* entradas utilizadas */
9     size_t capacidad;   /* entradas disponibles */
10 } TablaFrecuencias;
```

Listing 1: Estructura de la tabla de frecuencias

La inserción recorre linealmente la tabla comparando con `strcmp`, por lo que el algoritmo es de complejidad $O(n^2)$: con 904 palabras distintas, cada inserción puede costar hasta 904 comparaciones. Esta característica, heredada del programa original, resulta determinante para el experimento, ya que genera suficiente carga de cómputo como para que la paralelización tenga un efecto medible.

3.2. Versión secuencial

Un único flujo de ejecución recorre el texto carácter por carácter, delimita cada palabra y la acumula en la tabla:

```
1 for (i = inicio; i < fin; i++) {
2     unsigned char actual = (unsigned char)texto[i];
3
4     if (es_caracter_de_palabra(actual)) {
5         palabra[longitud++] = tolower(actual); /* continua la palabra */
6     } else if (longitud > 0) {
7         palabra[longitud] = '\0'; /* termino: registrarla */
8         agregar_palabra(tabla, palabra, 1);
9         longitud = 0;
10    }
11 }
12 if (longitud > 0) { /* si el texto no termina en separador */
13     palabra[longitud] = '\0';
14     agregar_palabra(tabla, palabra, 1);
15 }
```

```
15 }
```

Listing 2: Núcleo del conteo, idéntico en ambas versiones

3.2.1. Modificaciones respecto al Corto 02

La lógica de conteo se conservó sin cambios. Se realizaron tres modificaciones, todas ellas necesarias para poder medir el tiempo de forma comparable:

- (a) **Separación de la E/S respecto del cómputo.** La versión original leía con `fgetc()` carácter por carácter *mientras* contaba. Ahora el archivo se carga íntegramente en memoria *antes* de iniciar el cronómetro. De no hacerlo, la medición incluiría el acceso a disco en lugar de reflejar únicamente el cómputo.
- (b) **Aislamiento del conteo en una función parametrizada.** `contar_palabras(texto, inicio, fin, tabla)` opera sobre un rango. La versión secuencial la invoca una vez sobre el texto completo; en la paralela, cada hilo la invoca sobre su bloque. Se trata literalmente de **la misma función en ambas versiones**, lo que garantiza que las diferencias de tiempo provengan del paralelismo y no de un cambio de algoritmo.
- (c) **Parámetro veces en agregar_palabra.** La versión secuencial siempre pasa el valor 1. El parámetro habilita la fusión de conteos parciales en la versión paralela, donde es necesario acumular de una sola vez varias apariciones.

3.3. Versión paralela

3.3.1. Estrategia de descomposición

La descomposición es **por datos**: el texto se divide en N bloques contiguos y cada hilo procesa uno. La figura 1 presenta el flujo completo, incluyendo la delimitación de la región medida.

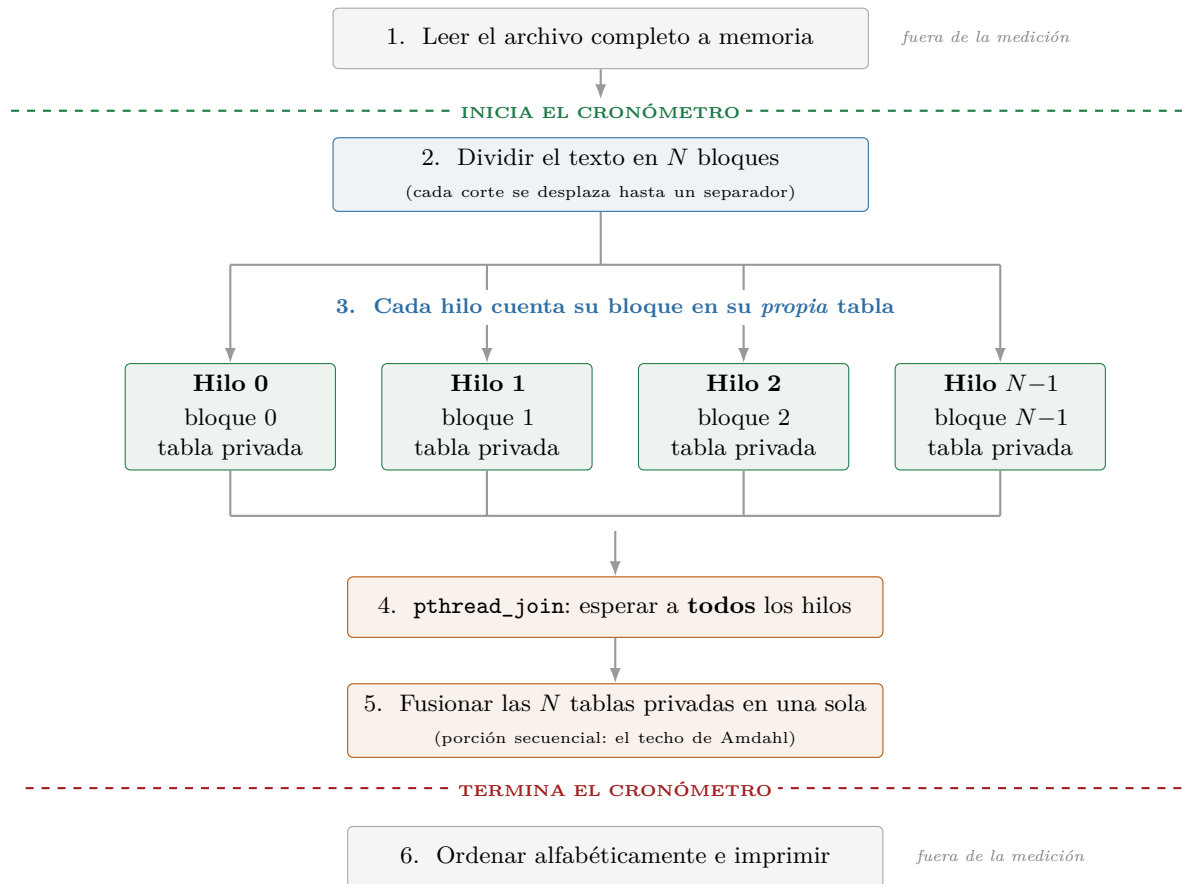


Figura 1: Flujo de la versión paralela. Únicamente la región comprendida entre las líneas punteadas forma parte de la medición.

3.3.2. División del texto sin fragmentar palabras

Si los cortes se calcularan por simple división aritmética, uno de ellos podría caer en medio de una palabra, que quedaría partida entre dos hilos y produciría dos tokens inexistentes. Para evitarlo, cada corte se desplaza hacia adelante hasta encontrar un separador:

```

1 corte = tamano * (i + 1) / num_hilos;
2
3 /* Avanzar hasta el siguiente separador para no partir palabras. */
4 while (corte < tamano && es_caracter_de_palabra((unsigned char)texto[corte])) {
5     corte++;
6 }
  
```

Listing 3: Ajuste de los cortes a frontera de palabra

Como consecuencia, los bloques no resultan exactamente del mismo tamaño. Este desbalance residual constituye una de las fuentes de *overhead* identificadas en la sección 6.6.

3.3.3. Tablas privadas por hilo

Cada hilo acumula sus resultados en una tabla a la que ningún otro hilo accede:

```

1 typedef struct {
2     const char *texto;
3     size_t inicio, fin;          /* su bloque: [inicio, fin) */
4     TablaFrecuencias tabla;      /* tabla privada del hilo */
5     int ok;
6 } TareaHilo;

```

Listing 4: Descriptor de tarea asignado a cada hilo

Esta es la decisión de diseño más determinante del programa. La alternativa —una única tabla compartida protegida por un *mutex*— habría sido contraproducente: dado que prácticamente toda la ejecución consiste en insertar palabras en la tabla, los hilos permanecerían la mayor parte del tiempo esperando el candado. El programa quedaría serializado en la práctica y, sumando el costo de adquirir y liberar el *mutex* millones de veces, resultaría previsiblemente **más lento que la versión secuencial**.

Al asignar datos privados a cada hilo, **la fase paralela no contiene ni un solo lock**. Todo el costo de coordinación se concentra en un único punto: la reducción final. Esto convierte al programa en un caso particularmente claro para observar la ley de Amdahl.

3.3.4. Sincronización y reducción

```

1 for (i = 0; i < num_hilos; i++) {
2     pthread_join(hilos[i], NULL);      /* esperar a que todos terminen */
3 }
4
5 /* Reduccion: volcar las tablas 1..N-1 sobre la del hilo 0. */
6 for (i = 1; i < num_hilos; i++) {
7     for (j = 0; j < tareas[i].tabla.cantidad; j++) {
8         agregar_palabra(&tareas[0].tabla,
9                         tareas[i].tabla.entradas[j].palabra,
10                        tareas[i].tabla.entradas[j].frecuencia);
11     }
12 }

```

Listing 5: Sincronización mediante join y reducción de las tablas

La reducción la ejecuta **un solo hilo** mientras los restantes ya han finalizado y permanecen ociosos. Constituye la porción estrictamente secuencial del algoritmo y, adicionalmente, **su costo crece con N** : con 2 hilos debe fusionarse una tabla; con 4 hilos, tres.

4. Metodología de medición

4.1. Delimitación de la región medida

Se empleó `clock_gettime` con el reloj `CLOCK_MONOTONIC`, que ofrece resolución de nanosegundos y, a diferencia del reloj de pared, no retrocede ante ajustes de hora del sistema.

Cuadro 2: Delimitación de la región medida en ambas versiones

	Versión secuencial	Versión paralela
Fuera (antes)	Lectura del archivo a memoria	Lectura del archivo a memoria
Dentro	Recorrido del texto y llenado de la tabla	División del texto Creación de los hilos Conteo parcial concurrente <code>pthread_join</code> Reducción de las tablas
Fuera (después)	Ordenamiento e impresión	Ordenamiento e impresión

La totalidad del *overhead* del paralelismo se encuentra dentro de la medición: el reparto del texto, la creación de los hilos, la espera en la barrera y la reducción final. No se midió únicamente la fase favorable.

4.2. Verificación de correctitud

Se comprobó que la versión paralela produce exactamente la misma tabla de frecuencias que la secuencial. La verificación está automatizada en el script `verificar.sh`, que devuelve código de salida 0 o 1.

Prueba 1: variación en la cantidad de hilos. Se comparó la salida completa (904 palabras con sus frecuencias) contra la referencia secuencial empleando **1, 2, 3, 4, 5, 7, 8, 16 y 32 hilos**. Se incluyeron deliberadamente números impares y no divisores del tamaño del archivo, con el fin de forzar cortes en posiciones irregulares. El resultado fue **idéntico en las nueve configuraciones**.

Prueba 2: casos borde. Cada caso se contrastó con 1, 2, 4 y 8 hilos.

Cuadro 3: Casos borde verificados

Caso	Bytes	Aspecto que ejercita	Resultado
Archivo vacío	0	Todos los bloques vacíos	Idéntico
Una sola palabra	4	Menos palabras que hilos	Idéntico
Solo espacios y tabuladores	7	Ausencia total de palabras	Idéntico
Dos palabras de un carácter	3	Más hilos que bytes	Idéntico
Palabras repetidas	16	Acumulación entre bloques	Idéntico
Mayúsculas y minúsculas	21	Normalización a minúscula	Idéntico
Sin salto de línea final	15	Palabra al borde del bloque	Idéntico

Los casos de 3 y 4 bytes son los más exigentes: con 8 hilos varios reciben bloques vacíos y el ajuste de frontera puede producir bloques de longitud cero. El programa los procesa sin fallar y sin perder ni duplicar palabras.

4.3. Justificación de las cinco ejecuciones

Los tiempos presentan una dispersión considerable entre corridas. Las cinco mediciones secuenciales sobre el archivo de 24 KB oscilaron entre 0.002 447 s y 0.004 826 s, es decir, **casi el**

doble de diferencia con el mismo código y el mismo archivo. Las causas son ajenas al programa: otros procesos compitiendo por el CPU, decisiones del planificador, variación dinámica de la frecuencia del procesador y efectos de calentamiento de caché. Promediar cinco corridas mitiga esta variabilidad.

5. Resultados experimentales

5.1. Tiempos de ejecución

Cuadro 4: Tiempos de ejecución sobre `archivo.txt` (24 530 bytes)

Núm.	T. secuencial (s)	T. paralela 2 hilos (s)	T. paralela 4 hilos (s)
1	0.004635	0.003043	0.002097
2	0.004826	0.002977	0.001848
3	0.003230	0.002034	0.001863
4	0.002752	0.001510	0.001490
5	0.002447	0.001376	0.001426
Promedio	0.003578	0.002188	0.001745

Cuadro 5: Tiempos de ejecución sobre `archivo_grande.txt` (981 240 bytes)

Núm.	T. secuencial (s)	T. paralela 2 hilos (s)	T. paralela 4 hilos (s)
1	0.078007	0.046513	0.024097
2	0.069338	0.047642	0.026287
3	0.077322	0.041301	0.022825
4	0.065508	0.038404	0.023097
5	0.067785	0.037561	0.027518
Promedio	0.071592	0.042284	0.024765

5.2. Cálculo del *speedup*

El *speedup* se define como el cociente entre el tiempo de la versión secuencial y el de la versión paralela:

$$S_p = \frac{T_{\text{secuencial}}}{T_{\text{paralelo}}} \quad (1)$$

Cuadro 6: *Speedup* por corrida y a partir de los tiempos promedio

archivo.txt (24 KB)			archivo_grande.txt (958 KB)		
Núm.	2 hilos	4 hilos	Núm.	2 hilos	4 hilos
1	1.5232	2.2103	1	1.6771	3.2372
2	1.6211	2.6115	2	1.4554	2.6377
3	1.5880	1.7338	3	1.8722	3.3876
4	1.8225	1.8470	4	1.7058	2.8362
5	1.7783	1.7160	5	1.8047	2.4633
Prom.	1.6353	2.0504	Prom.	1.6931	2.8909

Cálculo a partir de los tiempos promedio del archivo principal:

$$S_2 = \frac{0.003578}{0.002188} = \mathbf{1.6353} \quad (2)$$

$$S_4 = \frac{0.003578}{0.001745} = \mathbf{2.0504} \quad (3)$$

5.3. Cálculo de la *efficiency*

La *efficiency* expresa el aprovechamiento promedio de cada hilo:

$$E_p = \frac{S_p}{p} \quad (4)$$

Cuadro 7: *Efficiency* por corrida y a partir de los tiempos promedio

archivo.txt (24 KB)			archivo_grande.txt (958 KB)		
Núm.	2 hilos	4 hilos	Núm.	2 hilos	4 hilos
1	0.7616	0.5526	1	0.8386	0.8093
2	0.8105	0.6529	2	0.7277	0.6594
3	0.7940	0.4334	3	0.9361	0.8469
4	0.9113	0.4617	4	0.8529	0.7091
5	0.8892	0.4290	5	0.9023	0.6158
Prom.	0.8176	0.5126	Prom.	0.8466	0.7227

Cálculo a partir de los tiempos promedio del archivo principal:

$$E_2 = \frac{1.6353}{2} = \mathbf{0.8176} \quad (81.76 \%) \quad (5)$$

$$E_4 = \frac{2.0504}{4} = \mathbf{0.5126} \quad (51.26 \%) \quad (6)$$

5.4. Tabla final de resultados

Cuadro 8: Resumen consolidado de resultados

Archivo	Configuración	T. promedio (s)	Speedup	Efficiency
archivo.txt (24 KB)	Secuencial	0.003578	1.0000	100.00 %
	2 hilos	0.002188	1.6353	81.76 %
	4 hilos	0.001745	2.0504	51.26 %
archivo_grande.txt (958 KB)	Secuencial	0.071592	1.0000	100.00 %
	2 hilos	0.042284	1.6931	84.66 %
	4 hilos	0.024765	2.8909	72.27 %

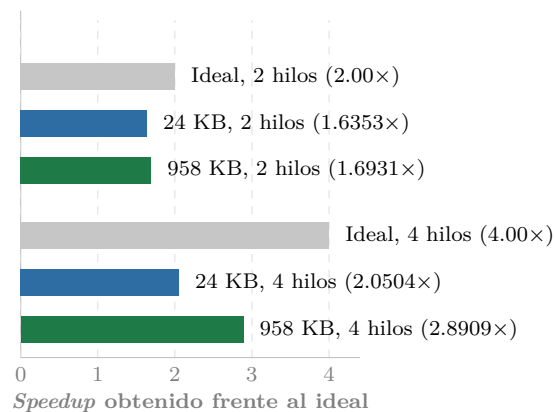


Figura 2: Comparación del *speedup* obtenido contra el *speedup* lineal ideal. La brecha se ensancha al pasar de 2 a 4 hilos, y es considerablemente menor con el archivo de mayor tamaño.

6. Análisis de resultados

6.1. ¿Con qué cantidad de hilos se obtuvo el mayor *speedup*?

Con **4 hilos** en ambos archivos. En `archivo.txt` el *speedup* pasó de 1.6353 (2 hilos) a 2.0504 (4 hilos), y en `archivo_grande.txt` de 1.6931 a 2.8909. El mejor resultado absoluto fue 2.8909× **con 4 hilos sobre el archivo de mayor tamaño.**

No obstante, el incremento de 2 a 4 hilos rindió considerablemente menos de lo esperado: duplicar la cantidad de hilos debería duplicar el *speedup*, pero en el archivo pequeño únicamente lo aumentó un 25 % (1.64 → 2.05) y en el grande un 71 % (1.69 → 2.89).

6.2. ¿Con qué configuración se obtuvo la mayor *efficiency*?

Con **2 hilos**: 81.76 % en el archivo pequeño y 84.66 % en el grande. En ambas pruebas la eficiencia con 2 hilos superó a la de 4 hilos. El valor más bajo correspondió a **4 hilos sobre el archivo pequeño (51.26 %)**, donde prácticamente la mitad de la capacidad de cómputo asignada se consumió en *overhead*.

Se observa aquí una tensión característica del cómputo paralelo: **la configuración más rápida (4 hilos) no es la más eficiente (2 hilos)**. Con 2 hilos se aprovecha mejor cada núcleo asignado; con 4 hilos se termina antes, pero a costa de recursos ociosos.

6.3. ¿El *speedup* aumentó proporcionalmente?

No. El crecimiento fue claramente **sublineal** y se alejó del ideal conforme se agregaron hilos.

Cuadro 9: Comparación con el *speedup* lineal ideal

Hilos	Speedup ideal	Obtenido (24 KB)	Obtenido (958 KB)
2	2.00	1.64 (82 % del ideal)	1.69 (85 % del ideal)
4	4.00	2.05 (51 % del ideal)	2.89 (72 % del ideal)

Con 2 hilos se alcanzó entre el 82 % y el 85 % del ideal, mientras que con 4 hilos únicamente entre el 51 % y el 72 %. La brecha respecto al *speedup* lineal **se ensancha** conforme aumenta el número de hilos, comportamiento que corresponde exactamente a lo que predice la **ley de Amdahl**: la porción secuencial del programa —en este caso la fusión de las tablas privadas— impone un techo al *speedup* alcanzable con independencia de cuántos hilos se agreguen.

6.4. ¿La *efficiency* aumentó o disminuyó?

Disminuyó de forma consistente en ambas pruebas:

- `archivo.txt`: 81.76 % → 51.26 % (**caída de 30.5 puntos**).
- `archivo_grande.txt`: 84.66 % → 72.27 % (**caída de 12.4 puntos**).

La razón es que el *overhead* —creación de hilos, reparto del texto y reducción final— **crece con el número de hilos**, mientras que el trabajo útil asignado a cada hilo **se reduce**. La relación entre trabajo útil y *overhead* empeora en cada incremento.

Resulta particularmente revelador comparar ambas caídas: con el archivo grande la pérdida de eficiencia fue **menos de la mitad** que con el pequeño, dado que existe considerablemente más trabajo real entre el cual distribuir el mismo costo fijo de coordinación.

6.5. Factores que pudieron afectar el tiempo de ejecución

1. **Tamaño del archivo** (factor determinante). Fue el efecto más marcado del experimento. Con 24 KB el trabajo asignado a cada hilo es tan reducido que el costo de crear hilos y fusionar tablas pesa casi tanto como el conteo mismo ($E_4 = 51\%$). Al pasar a 958 KB —cuarenta veces más texto— la eficiencia con 4 hilos ascendió al 72 % **sin modificar una sola línea de código**.
2. **Arquitectura heterogénea del procesador**. El Apple M5 dispone de 4 núcleos de rendimiento y 6 de eficiencia, que no ofrecen el mismo desempeño. Si el planificador ubica algún hilo en un núcleo E, dicho hilo tarda más y los restantes deben **esperarlo en el `pthread_join`**. El tiempo total lo determina el hilo más lento, no el promedio.

3. **Variabilidad del sistema operativo.** Los tiempos de una misma configuración varían de forma apreciable, con diferencias de hasta el doble entre la corrida más lenta y la más rápida, atribuibles a otros procesos, al planificador y a los cambios dinámicos de frecuencia.
4. **Efectos de caché y calentamiento.** Las primeras corridas resultaron sistemáticamente más lentas (0.004 635 s la primera frente a 0.002 447 s la quinta).
5. **Desbalance de carga.** Los bloques se reparten por bytes y no por palabras, de modo que los hilos no reciben exactamente la misma cantidad de trabajo.
6. **Complejidad $O(n^2)$ del algoritmo de tabla.** Cada inserción realiza búsqueda lineal. Este costo cuadrático es, paradójicamente, lo que hace que la paralelización resulte provechosa: proporciona suficiente cómputo que distribuir.

6.6. Fuentes de *overhead* identificadas

Cuadro 10: Fuentes de *overhead*, ordenadas por impacto

Fuente	Naturaleza del costo
Reducción de las tablas (la principal)	Es 100 % secuencial y crece con N : con 4 hilos deben fusionarse 3 tablas de hasta 904 palabras, y cada inserción realiza búsqueda lineal $O(904)$. Constituye el techo de Amdahl del programa.
Creación y destrucción de hilos	Cada <code>pthread_create</code> cuesta decenas de microsegundos. Sobre un trabajo total de 3.5 ms, cuatro hilos representan un costo fijo no despreciable.
Barrera implícita del join	El tiempo lo determina el hilo más lento; los restantes quedan ociosos. Se agrava con el desbalance de carga y con los núcleos heterogéneos.
Trabajo duplicado en memoria	Con 4 hilos coexisten 4 tablas de hasta 904 entradas: cuádruple uso de memoria y cuádruples llamadas a <code>malloc/realloc</code> .
Contención en el asignador	El asignador del sistema emplea <i>locks</i> internos; varios hilos solicitando memoria simultáneamente se serializan de forma parcial.
Cálculo de los cortes	Costo menor, pero corresponde a trabajo que la versión secuencial no realiza.
False sharing	Estructuras <code>TareaHilo</code> contiguas pueden compartir línea de caché, provocando invalidaciones cruzadas.

Conviene destacar que la fuente de *overhead* que se **evitó** explica también los buenos resultados con 2 hilos: al asignar a cada hilo una tabla privada en lugar de una tabla compartida con *mutex*, la fase paralela quedó libre de contención.

6.7. ¿Agregar más hilos garantiza siempre un mejor rendimiento?

No. Los datos obtenidos lo evidencian en tres niveles:

(a) **El rendimiento mejora, pero cada vez menos.** Al duplicar de 2 a 4 hilos en el archivo pequeño, el tiempo descendió de 0.002 188 s a 0.001 745 s: **se duplicaron los recursos para obtener apenas un 20 % de mejora**. El segundo par de hilos aportó una fracción mínima de lo que aportó el primero.

(b) **La eficiencia se degrada severamente.** Pasar de 2 a 4 hilos redujo la eficiencia del 81.76 % al 51.26 % en el archivo pequeño. En términos prácticos, con 4 hilos **casi la mitad de la capacidad de cómputo asignada se desperdició** en coordinación.

(c) **Existen ejecuciones donde 4 hilos resultó más lento que 2.** No se trata únicamente de un promedio que se degrada. En el archivo pequeño, la corrida 5 registró 0.001 376 s con 2 hilos frente a 0.001 426 s con 4, y la corrida 4 fue prácticamente un empate (0.001 510 s frente a 0.001 490 s). Agregar hilos en esos casos **no aportó absolutamente nada**.

Conclusión. El número óptimo de hilos depende de la relación entre trabajo útil y *overhead*, no de la cantidad de núcleos disponibles. El procesador empleado dispone de 10 núcleos, pero con `archivo.txt` la eficiencia ya se situaba por debajo del 52 % con 4 hilos; emplear 10 hilos sobre 24 KB de texto sería contraproducente, dado que el costo de crear y fusionar diez tablas superaría el ahorro obtenido en el conteo.

7. Conclusiones

1. La paralelización del conteo de frecuencia **sí produce una mejora real y medible**: hasta $2.89\times$ con 4 hilos, manteniendo resultados idénticos a la versión secuencial en todas las configuraciones probadas.
2. El **mayor *speedup*** se obtuvo con 4 hilos, pero la **mayor *efficiency*** con 2 hilos. Ambas métricas responden a criterios de optimización distintos y no deben confundirse.
3. El *speedup* crece de forma **sublineal** y la *efficiency* **decrece** al agregar hilos, comportamiento explicado por la ley de Amdahl: la reducción de las tablas es estrictamente secuencial y su costo aumenta con el número de hilos.
4. El **tamaño del problema resultó ser el factor más influyente**. El mismo código con 4 hilos pasó del 51 % al 72 % de eficiencia únicamente al incrementar el tamaño del archivo. El beneficio del paralelismo depende de que exista trabajo suficiente para amortizar el costo de coordinación.
5. La decisión de emplear **tablas privadas por hilo** en lugar de una tabla compartida con *mutex* fue determinante. Concentra todo el costo de sincronización en un único punto —la reducción— en lugar de distribuirlo a lo largo de toda la ejecución.

8. Reproducción de los resultados

```
1 make                # compila ambas versiones
2 make verificar      # verifica que la salida paralela sea identica
3 make benchmark      # 5 corridas por configuracion, speedup y efficiency
4
5 # Compilacion manual:
6 gcc -O2 -Wall        -o conteo_secuencial conteo_frecuencia_secuencial.c
7 gcc -O2 -Wall -pthread -o conteo_paralelo  conteo_frecuencia_paralelo.c
8
9 # Ejecucion individual:
10 ./conteo_secuencial archivo.txt
11 ./conteo_paralelo   archivo.txt 4
12 ./conteo_paralelo   archivo.txt 4 --silencioso # solo el tiempo
13
14 # Experimento complementario:
```

```
15 make benchmark ARCHIVO=archivo_grande.txt REPS=5
```

Listing 6: Comandos para reproducir el experimento

Los tiempos crudos de cada corrida se encuentran en los archivos `resultados_archivo.csv` y `resultados_grande.csv`.